

P³-LLM: An Integrated NPU-PIM Accelerator for Edge LLM Inference Using Hybrid Numerical Formats

Yuzong Chen^{*}, Chao Fang[†], Xilai Dai^{*}, Yuheng Wu[‡],
Thierry Tamba[‡], Marian Verhelst[†], Mohamed S. Abdelfattah^{*}

^{*}*Department of Electrical and Computer Engineering, Cornell University*

[†]*EAST-MICAS, KU Leuven*

[‡]*Department of Electrical Engineering, Stanford University*

^{*}{yc2367, mohamed}@cornell.edu

[†]{chao.fang, marian.verhelst}@esat.kuleuven.be

[‡]{yuhengwu, ttamba}@stanford.edu

Abstract—The substantial memory bandwidth and computational demands of large language models (LLMs) present critical challenges for efficient inference. To tackle this, the literature has explored heterogeneous systems that combine neural processing units (NPUs) with DRAM-based processing-in-memory (PIM) for LLM acceleration. However, the high-precision PIM compute units incur significant area and power overhead in DRAM technology, limiting the effective computation throughput. In this paper, we introduce P³-LLM, a novel NPU-PIM integrated accelerator for edge LLM inference. Our approach is threefold: First, we propose a flexible mixed-precision quantization scheme, which leverages hybrid numerical formats to quantize different LLM operands with high compression efficiency and minimal accuracy loss. Second, we architect an efficient PIM accelerator for P³-LLM, featuring enhanced compute units to support hybrid numerical formats. Our careful choice of numerical formats allows to co-design low-precision PIM compute units that significantly boost the computation throughput under iso-area constraints. Third, we optimize the low-precision dataflow of different LLM modules by applying operator fusion to minimize the overhead of runtime dequantization. Evaluations on diverse LLMs and tasks demonstrate that P³-LLM achieves higher accuracy than state-of-the-art KV-cache quantization and weight-activation quantization algorithms. Combining the proposed quantization scheme with low-precision PIM architecture co-design, P³-LLM yields an average of 4.9 $\times$, 2.0 $\times$, and 3.4 $\times$ speedups over state-of-the-art LLM accelerators HBM-PIM, Ecco, and Pimba, respectively. Code is available at <https://github.com/yc2367/P3-LLM>.

I. INTRODUCTION

Large language models (LLMs) have revolutionized various machine learning tasks such as text generation [38], [84], [90], image understanding [3], [65], and logical reasoning [11], [68], [73]. Besides large-scale cloud serving, LLMs are becoming increasingly prevalent in diverse edge scenarios such as chatbot interaction, autonomous driving, and mobile assistant [50], [52], [60], [66], [92]. However, the intelligence of LLMs comes at the cost of substantial computation and memory demands, imposing a significant bottleneck for low-cost deployment, particularly in edge scenarios with limited hardware resources.

To enhance the performance and efficiency of edge LLM deployment, heterogeneous LLM acceleration based on neural processing unit (NPU) and DRAM-based processing-in-memory (PIM) has been actively explored [27], [44], [48], [49], [51], [77], [81]. An integrated NPU-PIM accelerator leverages NPU and PIM to accelerate two distinct LLM inference stages, prefilling and decoding, respectively. During the

prefilling stage, the LLM performs compute-intensive general matrix-matrix multiplication (GEMM) that can be effectively accelerated by NPU with high computational parallelism [39]. While during the decoding stage, the LLM performs memory-intensive general matrix-vector multiplication (GEMV) that is well-suited for PIM with much higher bandwidth than conventional DRAM [40], [41]. Despite PIM’s great promise in accelerating LLM inference, the PIM compute units (PCUs) designed for high-precision (e.g. FP16) arithmetic suffer from significant area and power overhead due to the much lower transistor density in DRAM technology [13]. This PCU overhead severely limits the computation throughput of PIM, restricting its practical speedup mainly to single-batch inference on early-generation LLMs, whose multi-head attention and feed-forward layers exhibit an arithmetic intensity of one during decoding [67], [84]. Recently, low-batch edge LLM inference has gained much popularity [50], and state-of-the-art (SoTA) LLMs have adopted grouped-query attention (GQA) with arithmetic intensities larger than one [38], [64], [66], [90]. This trend necessitates new PCU design with higher throughput, while remaining within DRAM area constraints.

One effective solution to alleviate the cost of compute units is quantization. By reducing the operand bit-width, quantization not only decreases the memory footprint, but also enables faster and more area-efficient computation on low-precision hardware. Depending on the target quantized operands, LLM quantization algorithms can be broadly classified into three categories: (1) *weight-only* quantization [17], [52], [78]; (2) *KV-cache-only* quantization [28], [42], [61]; (3) *weight-activation*¹ quantization [2], [8], [16], [29], [46], [47], [53]. Among them, weight-only and KV-cache-only quantization are known to achieve near-lossless accuracy at 4-bit precision. However, since all other operands remain in FP16, these two methods offer limited compression and still require expensive hardware units for computation [6], [31], [75]. In contrast, weight-activation quantization significantly reduces both memory footprint and hardware computation cost, yet applying it to enable cost-effective PIM for LLM inference still poses several challenges.

On the algorithm side, SoTA weight-activation quantization algorithms like QuaRot [2] and QoQ [53] primarily use the

¹Here, we mainly refer activation quantization as quantizing both input activations and KV-cache.

conventional integer format, and rely on calibration datasets to approximate the dynamic behavior of activations in a static offline manner, thereby struggling to maintain good accuracy for other datasets due to overfitting. On the hardware side, adopting low-precision PCUs for LLM inference remains an open research problem in the architecture community, mainly because of the challenges in balancing memory saving, model accuracy, and hardware efficiency. MANT [29] and Ecco [8] propose aggressive quantization methods and co-designed NPU accelerators, but suffer from low computational efficiency due to the complicated numerical formats and encoding to represent quantized operands. Thus, these proposals cannot be directly applied to PIM with stringent area constraints. While a recent PIM work, Pimba [44], adopts the 8-bit microscaling format to improve the area efficiency of PCUs, it overemphasizes preserving model accuracy with 8-bit KV-cache-only quantization, resulting in low memory savings for edge LLM inference. Furthermore, offloading the self-attention module of LLMs to low-precision PCU introduces additional challenges, such as the difficulty of accurately quantizing attention-scores.

To overcome the above limitations, we propose P³-LLM, an NPU-PIM integrated accelerator for low-Precision edge LLM inference. Given the substantial memory and computation demands of LLMs, P³-LLM employs mixed-precision quantization with 4-bit weights and KV-cache to achieve high compression, as well as 8-bit activations and attention scores to reduce computational complexity. Our key algorithmic innovation is an operand-dependent quantization scheme that leverages hybrid numerical formats to aggressively quantize different LLM operands with minimal accuracy loss. For KV-cache, we propose a dynamic, input-aware smoothing strategy that mitigates outliers without calibration datasets, thus enabling accurate and efficient 4-bit integer quantization while preventing overfitting. For attention-scores, we introduce an unsigned 8-bit floating-point format with 4-bit exponent and 4-bit mantissa (FP8-S0E4M4) to offer superior numerical fidelity. For activations, we explore the accuracy-efficiency tradeoffs among different 8-bit quantization options, and identify the direct FP8-E4M3 cast as the optimal choice. Our quantization algorithm is further equipped with a low-precision PIM co-design, which features high-throughput and area-efficient PCUs to flexibly support hybrid numerical formats. Finally, we architect low-precision dataflow of different LLM modules by applying operator fusion to minimize the overhead of runtime dequantization.

The main contributions of this paper are summarized below:

- 1) We introduce P³-LLM, an algorithm-hardware co-design solution with carefully optimized low-precision dataflow to unleash the potential of PIM for edge LLM inference.
- 2) We propose a novel LLM quantization scheme that employs hybrid numerical formats to achieve an excellent trade-off between model accuracy, memory footprint, and computational efficiency.
- 3) We design a high-throughput and area-efficient PCU architecture to accelerate the proposed quantization scheme, while remaining within the PIM area constraints.
- 4) Through comprehensive evaluation, we demonstrate that P³-LLM achieves higher accuracy than SoTA LLM

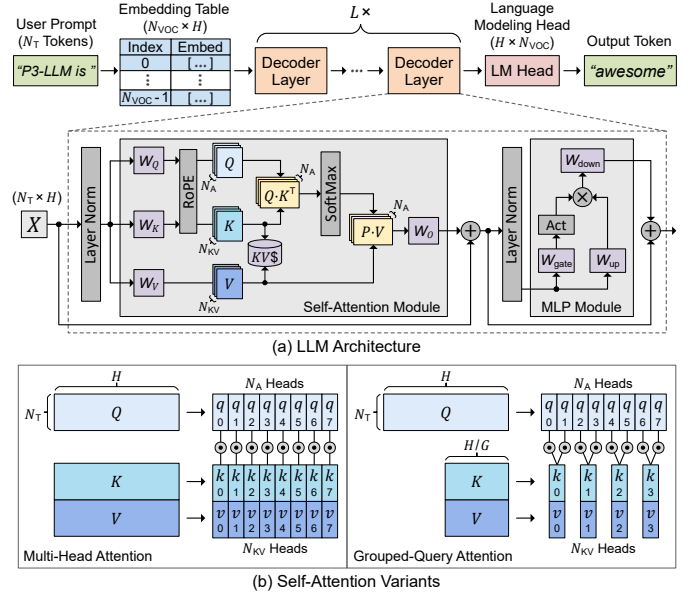


Fig. 1: Illustration of LLM architecture.

quantization algorithms Oaken [42], QuaRot [2], and QoQ [53], while offering an average of $4.9\times$, $2.0\times$, and $3.4\times$ speedups over the SoTA LLM accelerators HBM-PIM [49], Ecco [8], and Pimba [44], respectively.

II. BACKGROUND

A. Architecture of Large Language Models (LLMs)

As depicted in Fig. 1(a), mainstream LLMs have a series of decoder layers in addition to an input embedding table and an output language modeling (LM) head. During the prefilling stage of inference, the LLM receives a user prompt containing N_T tokens, and converts it to an input matrix through an embedding table of size $N_{VOC} \times H$, where N_{VOC} is the vocabulary size and H is the hidden dimension size. The input matrix is processed by $L \times$ decoder layers, followed by the LM head at the end to produce the first output token. During the decoding stage, the LLM takes this output token as input, and performs the same operation as in the prefilling stage to generate new tokens in an auto-regressive manner.

The decoder layer serves as the fundamental component in LLMs, consisting of a self-attention module and a multi-layer perceptron (MLP). The self-attention module begins with three linear layers (W_Q , W_K , W_V) to generate query, key, and value vectors, respectively. In recent LLM architectures [38], [84], [90], W_Q and W_K are usually followed by rotary position embedding (RoPE) [83], which encodes positional information into the query and key vectors through matrix rotation. The generated key and value vectors are also cached in off-chip memory for computation reuse during future decoding iterations, and are therefore referred to as the KV-cache. Then, each query and key-value vector is split into N_A and N_{KV} heads, respectively, where N_A is the number of attention heads and N_{KV} is the number of key-value heads. For every attention head, the query vectors are multiplied with the transposed key vectors ($Q \cdot K^T$), followed by a softmax function to calculate the attention-scores (P). The attention-scores are then multiplied with the value vectors ($P \cdot V$), and the results

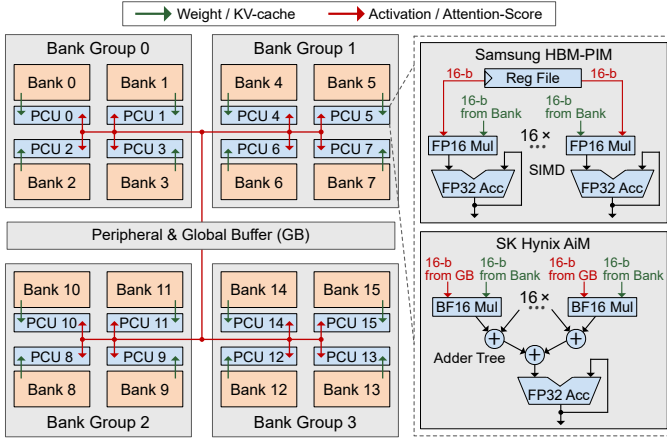


Fig. 2: Illustration of PIM architectures for LLM decoding acceleration.

are passed through a linear layer (W_O) to produce the attention output states. The MLP module contains three linear layers ($W_{gate}, W_{up}, W_{down}$) to produce the MLP output states.

Considering the KV-cache needs to be stored for every token, its capacity can become significant for long-context scenarios [4]. To mitigate this storage overhead, recent LLMs have adopted the GQA mechanism. As shown in Fig. 1(b), conventional multi-head attention has the number of attention heads equal to the number of key-value heads, i.e., $N_A = N_{KV}$. On the other hand, GQA partitions the N_A attention heads into $G = N_A / N_{KV}$ groups (two in this example), and different groups share the same key-value vectors, effectively reducing the KV-cache capacity by $G \times$.

B. Processing In-Memory (PIM) for LLM Acceleration

DRAM-based PIM has become a promising solution to accelerate the decoding stage of LLM inference given its higher internal bandwidth tailored for memory-bound operations [27], [44], [48], [49], [51], [77], [81]. Fig. 2 illustrates the architectures of two commercially available PIM devices: Samsung’s HBM-PIM [49] and SK Hynix’s Accelerator-in-Memory (AiM) [48]. The left part of Fig. 2 shows a PIM channel consisting of 16 banks organized into 4 bank groups. One PIM compute unit (PCU) is placed near each DRAM bank to perform efficient GEMV operations by leveraging the abundant bank-level parallelism. Depending on area constraints, two banks may share the same PCU to amortize the area overhead [49]. During LLM decoding, the DRAM bank transfers weights / KV-cache data to the PCU in 256-bit granularity (i.e., 16×16 -bit operands). Meanwhile, the input vector is sent from the host to either the PCU register file in HBM-PIM or the global buffer in AiM.

As shown in the right part of Fig. 2, HBM-PIM and AiM have different implementations of the PCU microarchitecture. In HBM-PIM, the PCU contains a 16-way single-instruction-multiple-data (SIMD) MAC unit, and allows to exploit input reuse during GEMV by multiplying the same input element with 16 weights. On the other hand, the PCU of AiM uses the brain floating-point (BF16) format for data representation, and adopts an adder-tree-based design to exploit output reuse during GEMV. Despite their simplicity, the high-precision PCUs incur considerable area overhead, ranging from 20% to

27% of the DRAM die area [48], [49], primarily because the DRAM process has roughly $10 \times$ lower transistor density and fewer metal layers for routing compared to CMOS at the same technology node [13]. This overhead significantly constrains the achievable compute throughput of PIM, restricting its performance benefits mainly to single-batch inference and multi-head attention that do not exhibit data reuse.

C. LLM Quantization

Quantization is a widely used technique for cost-effective LLM acceleration. Consider a group of operands X and a list of quantization values Q , the quantized operand X_Q and dequantized operand $\tilde{X}$ can be calculated as follows:

$$\Delta = \frac{|X|_{\max}}{Q_{\max}}; X_Q = \text{Round}\left(\frac{X}{\Delta}, Q\right); \tilde{X} = X_Q \cdot \Delta \quad (1)$$

where Δ is the scaling factor, and $\text{Round}(x, Y)$ is a function that rounds a value x to the closest value in a set Y . This rounding process inevitably introduces error between the original and quantized operands. Numerous techniques have been proposed to reduce quantization error, such as mixed-precision quantization and custom numerical formats.

In mixed-precision domain, SoTA algorithmic solutions on weight-only [17], [52] and KV-cache-only [28], [61] quantization have demonstrated near-lossless accuracy at 4-bit precision. To further alleviate the computation and memory demands of LLMs, several studies have explored weight-activation quantization [2], [15], [53], [82]. Meanwhile, recent literature in the architecture community has explored custom numerical formats that can better adapt to the tensor distribution of LLMs [6], [16], [21], [29], [32], [47], [56], [69], [78], [79], [89]. In addition to research efforts, custom quantization formats have been widely adopted by industry. For instance, NVIDIA and AMD support the 8-bit floating-point format with two variants: 4-bit exponent 3-bit mantissa (FP8-E4M3) and 5-bit exponent 2-bit mantissa (FP8-E5M2) [69].

III. MOTIVATION

A. Memory Footprint vs. Quantization Sensitivity

Fig. 3(a) shows the memory breakdown of various LLMs at FP16, including Llama-2-7B [67], Llama-3.1-8B [64], Llama-3.2-3B [66], and Mistral-7B [38]. The batch size varies from 1 to 8 and the input context length is 4K, reflecting typical edge LLM inference scenarios [16], [20], [44], [50]. While model weights dominate the memory footprint at very low batch sizes, the KV-cache capacity significantly grows with increasing batch sizes. Conversely, activations and attention-scores have a smaller impact on memory footprint, since their memory can be released immediately after the associated GEMM/GEMV modules complete. Furthermore, Llama-2-7B requires much larger KV-cache than other LLMs, due to its usage of multi-head attention.

Fig. 3(b) demonstrates the impact of quantizing individual operands to low precision using the standard integer format, while maintaining other operands at FP16. We quantify the model performance of Llama-3.1-8B and Llama-3.2-3B on the C4 dataset [14] using the perplexity metric, where lower perplexity indicates better performance. Unlike the trend observed in memory footprint, activations and attention-scores exhibit

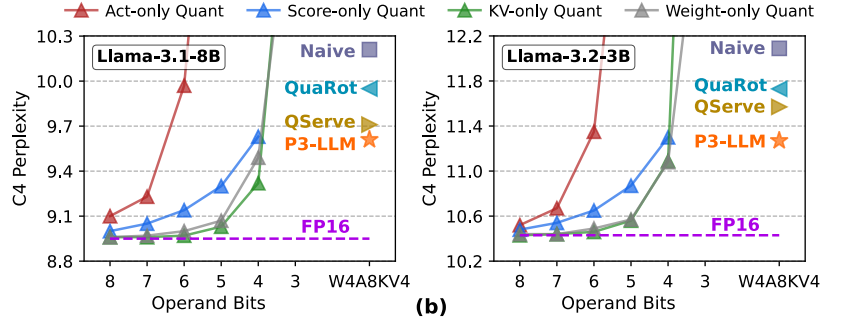
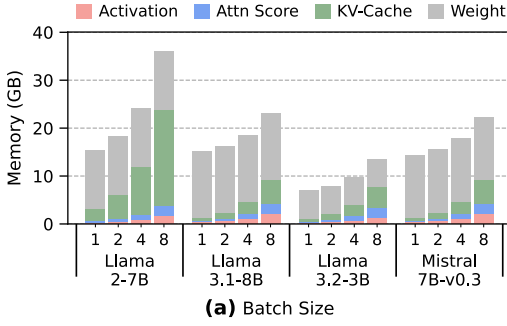


Fig. 3: Analysis of LLM operands: (a) Memory footprint of various LLMs at a 4K context length across different batch sizes. (b) Impact of quantization bit-width on the C4 perplexity (↓) of Llama-3.1-8B and Llama-3.2-3B. On the x-axis, operand bits represent the precision of quantizing each operand independently. The perplexity of baseline FP16 LLMs and different quantization methods under W4A8KV4 are also highlighted. Note that all W4A8KV4 methods use FP16 attention-scores, except for P³-LLM, which uses 8-bit attention-scores.

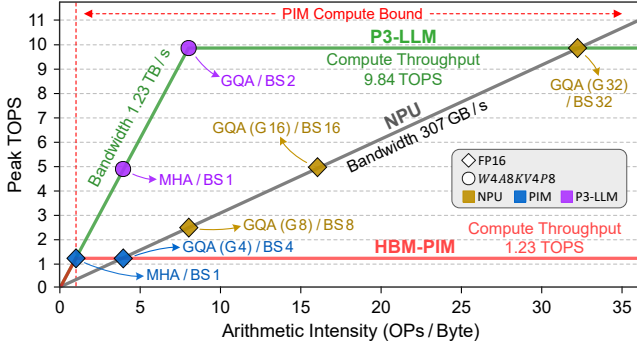


Fig. 4: Roofline analysis of different accelerators. The markers highlight the achievable throughput of various operators, including multi-head attention (MHA), grouped-query attention (GQA) with different group sizes (G), and linear layer with different batch sizes (BS).

larger sensitivity to quantization, leading to worse perplexity than weight and KV-cache under the same quantization bit-width. Furthermore, quantizing weight and KV-cache can maintain acceptable perplexity until 4-bit.

The above observations motivate our mixed-precision quantization scheme, namely W4A8KV4P8. In this scheme, weights (W) and KV-cache (KV) are quantized to 4 bits to maximize memory savings, while activations (A) and attention-scores (P) are retained at 8 bits to mitigate accuracy loss. Although attention-scores account for a small memory footprint, we argue that quantizing attention-scores further enhances hardware efficiency by allowing the $P \cdot V$ operation of the self-attention module to run on low-precision compute units. Unfortunately, the naïve W4A8KV4P8 quantization with the standard integer format results in large perplexity degradation, as highlighted in Fig. 3(b). To address this, P³-LLM proposes an operand-dependent W4A8KV4P8 quantization strategy using hybrid numerical formats (detailed in Section IV). This approach employs dedicated numerical format for each LLM operand to minimize their quantization error. As a result, P³-LLM achieves better model performance than SoTA W4A8KV4 integer quantization algorithms, QuaRot [2] and QoQ [53].

B. Limitations of Existing NPU-PIM Systems

Existing PIM solutions for LLM inference mainly adopt high-precision PCUs with limited computation throughput, thus facing challenges in low-batch decoding and GQA. To elucidate such limitations, we conduct roofline analysis on

TABLE I. P³-LLM vs. existing co-design solutions for quantized LLM acceleration. The precision “W α A β KV γ P δ ” stands for α -bit weights, β -bit activations, γ -bit KV-cache, and δ -bit attention-scores. By default, attention-scores are 16-bit.

Framework	Operand Precision	Memory Saving	Model Accuracy	Hardware Efficiency
BitMoD [6]	W4A16KV16	Medium	High	Medium
Oaken [42]	W16A16KV4	Medium	High	Low
MANT [29]	W4A8KV4	High	Medium	Medium
Ecco [8]	W4A8KV4	High	High	Low
Pimba [44]	W16A16KV8	Low	High	Medium
P ³ -LLM (Ours)	W4A8KV4P8	High	High	High

HBM-PIM supporting FP16 arithmetic [49], which has 4 $\times$ higher bandwidth than normal HBM during PIM operations. Fig. 4 reveals that the performance benefits of HBM-PIM gradually disappear as the batch size (BS) and GQA group size (G) approach 4, due to the limited computation throughput. On the other hand, NPU remains memory-bound even for moderate BS ≥ 16 , making it highly desirable to increase the compute throughput of PIM while exploiting its abundant bandwidth. To address this, P³-LLM leverages W4A8KV4P8 quantization to enable cost-effective LLM inference on compact, low-precision hardware. Compared to HBM-PIM, P³-LLM can integrate 4 $\times$ PCUs under iso-compute-area constraints. Furthermore, as we will discuss in Section V-D, the low-precision PCU enables 2 $\times$ higher operating frequency than the FP16 PCU, which effectively doubles the peak throughput. Thus, P³-LLM offers a superior roofline with 8 $\times$ higher throughput over HBM-PIM.

C. Limitations of Existing LLM Accelerator Co-Design

Numerous co-design solutions have been proposed by the architecture community to accelerate quantized LLMs, as summarized in Table I. BitMoD [6] and Oaken [42] focus on 4-bit quantization of weights and KV-cache, respectively. But since all other operands remain in FP16, these two approaches provide moderate memory saving and hardware efficiency. MANT [29] and Ecco [8] achieve greater memory saving and efficiency via W4A8KV4 quantization. Nevertheless, MANT exhibits noticeable accuracy drop under this aggressive quantization scheme since it does not address the outlier issue presented in activations and KV-cache. Whereas Ecco requires complicated online decompression of quantized operand back

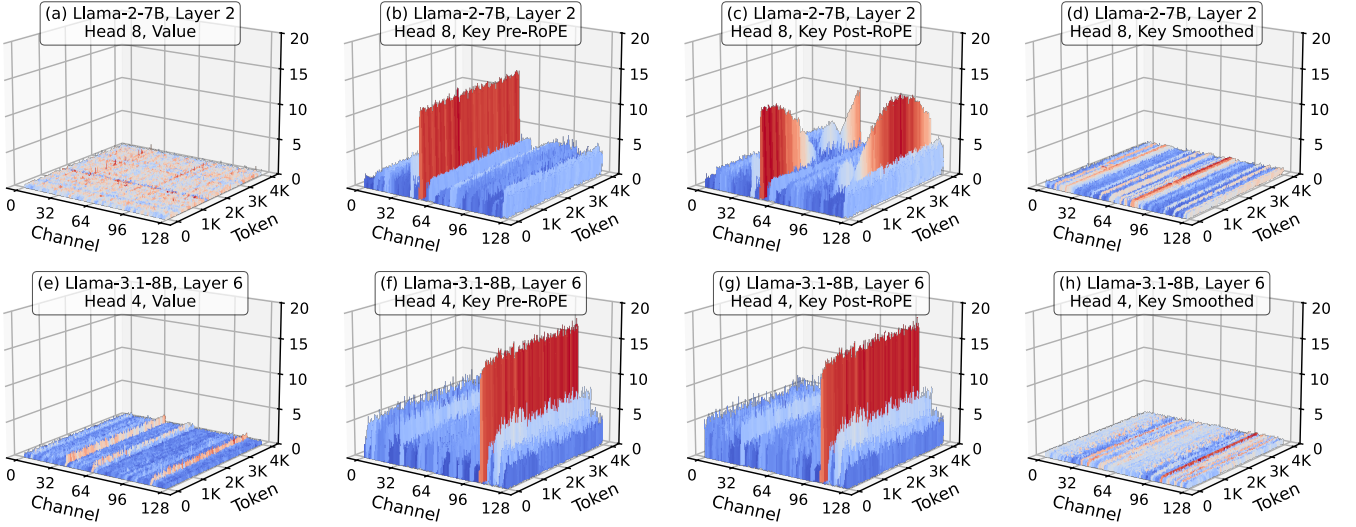


Fig. 5: KV-cache distribution (in absolute value) of Wikitext-2 dataset from representative layers and heads of Llama-2-7B and Llama-3.1-8B. The context length is 4K. (a)(e) The value cache shows no outlier pattern. (b)(f) The pre-RoPE key cache shows distinct outlier channels. (c) The post-RoPE key cache of Llama-2-7B contains much less structured outlier pattern. (g) The post-RoPE key cache of Llama-3.1-8B still exhibits distinct outlier channels. (d)(h) P³-LLM employs dynamic per-channel smoothing to eliminate the outlier channels of key cache.

to FP16 for computation, leading to considerable hardware overhead. Thus, although Ecco achieves high accuracy, its idea cannot be applied to designing low-precision PCUs for more efficient LLM decoding. On the other hand, Pimba [44] introduces a low-precision PIM architecture that exploits the 8-bit microscaling format [79] to mitigate the area overhead of PCUs. However, it leaves the linear layer unquantized, and the FP32 accumulation pipeline in the microscaling data path limits the overall hardware efficiency gain. Unlike these proposals, P³-LLM seeks to strike a balanced trade-off among memory footprint, model accuracy, and hardware efficiency. By assigning a dedicated precision and efficient numerical format to each LLM operand, P³-LLM minimizes both quantization error and memory usage. Our careful choice of hybrid numerical formats allows to co-design low-precision PIM that significantly boosts the computation throughput under iso-compute-area constraints.

IV. P³-LLM QUANTIZATION FRAMEWORK

This section details the P³-LLM quantization framework, the core of which is a mixed-precision W4A8KV4P8 scheme employing hybrid numerical formats. Furthermore, P³-LLM introduces a dynamic input-aware smoothing strategy to effectively suppress KV-cache outliers without overfitting.

A. KV-Cache Quantization

Profiling of KV-cache Distribution. Fig. 5 illustrates the KV-cache distribution from representative layers and heads of Llama-2-7B and Llama-3.1-8B, using Wikitext-2 dataset [63] with a context length of 4K. Fig. 5(a, e) show that the value cache has no outlier pattern, whereas Fig. 5(b, f) reveal that the key cache contains distinct outlier channels that pose significant challenges to quantization. These observations align with previous findings that certain fixed channels in activations exhibit large outliers [12], [88]. However, our profiling uncovers an additional important insight: the key cache of Llama-2-7B and Llama-3.1-8B exhibit drastically different patterns

after RoPE is applied. In Llama-2-7B, the post-RoPE key cache has much less structured outlier patterns. In contrast, the key cache of Llama-3.1-8B shows little difference before and after RoPE. The reason is that Llama-3 supports a much longer sequence length than Llama-2 (128K vs. 4K tokens). As described in Section II-A, RoPE encodes positional information into the key cache via matrix rotation—specifically, by multiplying each key vector with a rotation matrix whose angle is proportional to the token’s position index. When the maximum supported sequence length is long (e.g., 128K in Llama-3), the rotation angles are very small for typical input context lengths (e.g., 4K) targeting edge scenarios [16], [50], and RoPE has minimal impact on the key-cache distribution. Given these insights, we adopt pre-RoPE and post-RoPE key-cache quantization for Llama-2 and Llama-3, respectively, to take the advantage of structured outlier channels.

Dynamic Key-Cache Smoothing. To suppress outlier channels in the key cache (K), we propose a dynamic input-aware smoothing strategy. Assume K is of size $N_T \times H$, where N_T is the input context length and H is the hidden dimension size, the smoothed key cache K_S can be expressed as:

$$K_S[:, c] = \frac{K[:, c]}{\max(|K[:, c]|)} ; \text{ for } c \in [0, H) \quad (2)$$

Here, every key channel is divided by a smoothing factor, which is the per-channel absolute maximum, to lie in the numerical range of $[-1, 1]$. Consequently, the outlier channels are effectively suppressed as illustrated in Fig. 5(d, h). To apply dynamic smoothing to both prefilling and decoding stages, we calculate the smoothing factors by taking the per-channel absolute maximum of the key cache generated during prefilling. The smoothing factors are then saved and reused in the decoding stage to scale the newly generated key vectors. This approach provides two benefits. First, since every channel stores a smoothing factor shared by all tokens, the additional memory required is inversely proportional to the context length. Given that the context length can easily reach several

TABLE II. Wikitext-2 and C4 perplexity ($\downarrow$) using different 8-bit numerical formats to quantize attention-scores. KV-cache is quantized to INT4-Asym following Section IV-A. The context length is 4K.

Model Dataset	Llama-2-7B		Llama-2-13B		Llama-3.1-8B		Llama-3.2-3B	
	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4
FP16	5.15	6.67	4.60	6.08	5.95	8.59	7.41	10.18
INT8	5.19	6.71	4.65	6.12	6.06	8.71	7.55	10.31
FP8-E4M3	5.16	6.68	4.61	6.09	5.99	8.64	7.44	10.21
FP8-S0E4M4	5.15	6.67	4.60	6.08	5.95	8.59	7.42	10.18

hundreds, this memory overhead is $< 1\%$ and negligible. Second, the calculation of smoothing factors only involves the prefilling context and has no impact on the decoding stage. Our profiling on an A6000 GPU running Llama-3.1-8B reveals that this calculation takes < 5 millisecond in total across all layers, even under a context length of 32K. When considering a very stringent service-level objective in chatbot applications, with a 250-millisecond requirement on the time-to-first-token [97], the runtime overhead of dynamic smoothing is still $< 2\%$.

While per-channel smoothing has been explored by prior quantization works, our proposed strategy has two notable differences. First, prior methods often rely on a calibration dataset to determine the outlier threshold [42] or smoothing factor [53], [88] offline, which can lead to overfitting. In contrast, our method does not require any calibration dataset. Second, unlike prior works that focus exclusively on either pre-RoPE [28] or post-RoPE [53] key-cache quantization, we investigate both approaches by carefully analyzing the impact of RoPE on the resulting key cache distribution of different models. Section V-B discuss the impact of pre-RoPE and post-RoPE quantization on the co-designed dataflow of P³-LLM.

Quantization Format. Due to the dynamic nature of KV-cache, we employ 4-bit asymmetric integer (INT4-Asym) quantization, which offers superior computational efficiency. Furthermore, since the value cache shows no outlier pattern and the key cache has been greatly smoothed, their uniform distributions are well-suited to the integer format.

B. Attention-Score Quantization

In order to fully exploit the hardware efficiency of KV-cache quantization, it is essential to quantize additional operands, e.g., attention-scores, within the self-attention module. Although attention-scores have little impact on the overall memory footprint (see Section III-A), they play a critical role in the computation flow. Consider a low-precision PCU supporting 8-bit multiplication, if attention-scores remain in FP16, the PCU cannot be fully utilized to accelerate self-attention. Instead, the quantized value cache must be transferred to NPU and rely on the FP16 compute units to perform multiplication with attention-scores, thus diminishing the bandwidth advantages of PIM for LLM decoding. A straightforward solution is to quantize attention-scores with INT8 [95], but this introduces two drawbacks. First, INT8 attention-scores cause noticeable perplexity degradation as shown in Table II. Second, as we will demonstrate in Section IV-D, INT8 can lead to significant accuracy loss for activation quantization. Since attention-scores behave the same as activations during the MAC operation on hardware, it is desirable to identify a good numerical format that balances quantization accuracy and hardware complexity.

TABLE III. Wikitext-2 and C4 perplexity ($\downarrow$) of different weight-activation quantization methods. For activations, we compare INT8-based SmoothQuant (SQ) and direct FP8-E4M3 quantization. For weights, we examine 4-bit BitMoD. The context length is 4K.

Precision		2-7B		2-13B		3.1-8B		3.2-3B	
Weight	Activation	Wiki	C4	Wiki	C4	Wiki	C4	Wiki	C4
16	16	5.12	6.63	4.57	6.05	5.84	8.43	7.28	10.01
16	INT8-SQ	5.15	6.67	4.61	6.09	5.92	8.54	7.34	10.08
16	FP8-E4M3	5.12	6.63	4.58	6.05	5.85	8.46	7.31	10.03
4	INT8-SQ	5.37	6.95	4.76	6.27	6.36	9.12	7.87	10.73
4	FP8-E4M3	5.24	6.78	4.66	6.15	6.16	8.90	7.64	10.48

To address this, we propose an unsigned 8-bit floating-point format, FP8-S0E4M4, that contains a 4-bit mantissa and a 4-bit exponent with an exponent bias of -15 . There are two insights that drive our design choice. First, because the attention-scores are produced after softmax, its numerical range always lies between 0 and 1, eliminating the need for a sign bit in its encoding. Second, recall FP16 has a 5-bit exponent with a bias of -15 , which provides an exponent range of $[-14, 15]$. However, since attention-scores are always less than 1, there is no need to use positive exponent values, leading to an effective exponent range of $[-14, -1]$ with 14 distinct values. Thus, a 4-bit exponent is sufficient to represent the attention-score range, leaving 4 bits to store mantissa with better numerical fidelity. Table II reflects that FP8-S0E4M4 outperforms both INT8 and FP8-E4M3 for attention-score quantization, and achieves near-lossless model performance.

C. Weight Quantization

For weight quantization, we build upon an existing numerical format, BitMoD [6], which adaptively remaps the redundant negative zero encoding of FP4 to some pre-defined special values. On top of the basic FP4 quantization values $\{\pm 0, \pm 0.5, \pm 1, \pm 1.5, \pm 2, \pm 3, \pm 4, \pm 6\}$, BitMoD introduces four additional special values $\{\pm 5, \pm 8\}$, where one of them can be selected to replace the negative zero. By searching for the optimal special value for each group of weights, it reduces the quantization error compared to asymmetric integer quantization with minimal hardware overhead. Notably, since weights and KV-cache are mapped to the same operand on the MAC hardware, a decoder is required to accommodate the numerical formats of both weights and KV-cache (i.e., BitMoD and INT4-Asym), which we discuss in Section V-A.

D. Activation Quantization

Similar to the key cache, activations are also known to exhibit outlier channels [12], which makes standard integer quantization error-prone. To reduce the quantization error of integer activations, prior works mainly leverage two techniques to suppress outliers. The first is Hadamard transformation, which applies a Hadamard rotation matrix to activations before performing quantization [2], [59]. However, the on-line Hadamard transformation can bring considerable runtime overhead [8]. The second is smoothing, which migrates the quantization difficulty of outlier activation channels to the corresponding weight channels [88]. Unfortunately, the migration of quantization difficulty can significantly increase the error of low-precision weights.

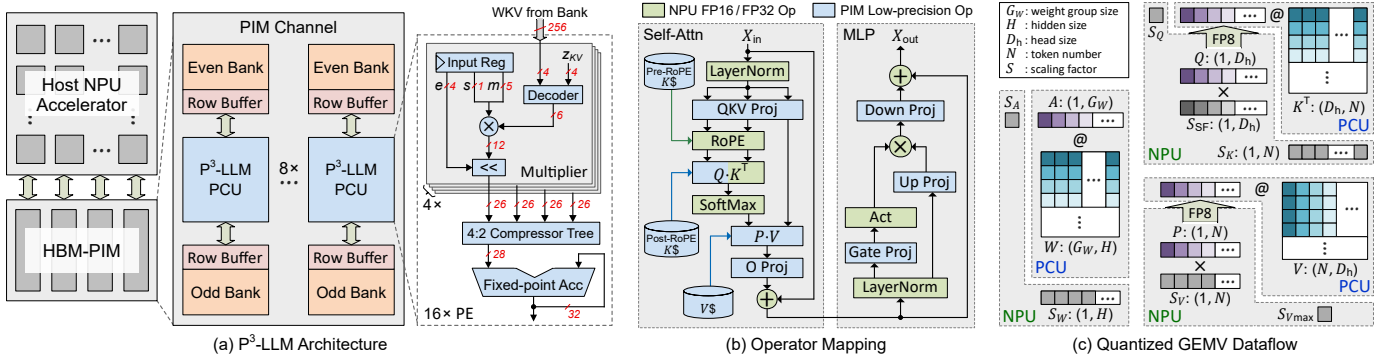


Fig. 6: (a) The architecture of P³-LLM. (b) Operator mapping on P³-LLM during decoding. (c) The quantized dataflow of three GEMV operations: weights @ activations, query @ key, attention-score @ value. For clarity, we use "@" and "x" to denote GEMV and element-wise multiplication, respectively.

To address these issues, we propose leveraging an 8-bit format that remains well-suited for activations even in the presence of outliers. Inspired by prior studies [85], [86], we explore FP8 as an alternative format for activation quantization without Hadamard transformation and smoothing. We find that the standard FP8-E4M3 format can achieve near-lossless accuracy for quantizing activations, owing to its wide numerical range. Table III shows the perplexity of per-token activation quantization using FP8-E4M3 and SmoothQuant-based [88] INT8. SmoothQuant causes large perplexity loss under 4-bit weights. This is expected, because weights are already difficult to quantize at 4 bits, yet SmoothQuant migrates additional difficulty to weights. On the other hand, FP8-E4M3 significantly outperforms SmoothQuant as its large numerical range can accommodate outliers without affecting weight quantization.

V. P³-LLM ARCHITECTURE

This section outlines the architectural details of P³-LLM, which features: (a) a high-throughput and area-efficient PCU co-designed to support hybrid numerical formats, (b) an efficient operator mapping that maps memory-intensive operators to the low-precision PIM hardware during decoding, (c) a quantized matrix multiplication flow that applies operator fusion to minimize the runtime overhead of dequantization.

A. P³-LLM Microarchitecture

To support the proposed W4A8KV4P8 scheme with hybrid numerical formats in PIM, we design an efficient PCU on top of HBM-PIM [49]. Fig. 6(a) left shows the P³-LLM architecture containing a host NPU and an HBM-PIM. One PIM channel contains 8 PCUs, each of them is shared by 2 banks to amortize the area overhead. During the decoding stage, the NPU sends activations and attention-scores to the input register of PCU, while weights and KV-cache are read from DRAM banks to perform MAC operations. Given that weights and KV-cache are quantized to 4 bits in P³-LLM, we choose to equip each PCU with 64 multipliers to match the 256-bit column access granularity of HBM, effectively quadrupling the compute throughput compared to the baseline HBM-PIM with 16 FP16 multipliers. However, scaling up the number of multipliers makes the SIMD-based design of HBM-PIM inefficient, as every multiplier must be paired with a 32-bit accumulator, leading to substantial register overhead.

To address this, we choose to maintain the same number of output registers as in HBM-PIM, and increase the input bandwidth to match the computational throughput. Fig. 6(a) right shows the proposed PCU microarchitecture containing 16 processing elements (PEs). The PCU computes a $1 \times 4 \times 16$ GEMV tile², which consumes 4 8-bit inputs, 64 4-bit weights, and produces 16 32-bit outputs. On the other hand, the baseline HBM-PIM's PCU computes a $1 \times 1 \times 16$ GEMV tile with one FP16 inputs, 16 FP16 weights, and 16 FP32 outputs. Thus, compared to the baseline HBM-PIM, P³-LLM only requires 16-bit additional input register per PCU with minimal overhead. Inside the low-precision PCU, every PE computes a 4-way dot product, where the multiplication involves a 6-bit fixed-point multiplier that multiplies the signed (s) input mantissa (m) with the decoded weight or KV-cache. The multiplier bit-width is determined in the following ways: First, computing the attention-score requires a 5-bit mantissa (including the hidden bit), and the activation requires a sign bit, resulting in a total of 6 bits. Second, since KV-cache adopts INT4-Asym quantization, its precision is 5-bit after adding a 4-bit zero point (z_{KV}). On the other hand, weights are encoded with the BitMoD data type [6], which requires 6 bits in fixed-point representation. After multiplication, the 4-bit input exponent (e) is used to shift the product. The results from 4 multipliers are reduced through a 4:2 compressor tree, followed by 32-bit fixed-point accumulation. Thus, the proposed PE eliminates the area-hungry components such as high-precision multipliers and exponent alignment modules in the conventional FP16/FP32 MAC unit.

B. PIM Offloading Opportunities with P³-LLM

The proposed W4A8KV4P8 quantization allows memory-bound matrix operations to be efficiently executed on the low-precision PCU, as illustrated in Fig. 6(b). During the decoding stage, the host NPU performs element-wise operations such as RoPE and Softmax in high precision, while GEMM/GEMV operators such as linear layers are offloaded to PIM. One exception is $Q \cdot K^T$, whose mapping depends on whether the key cache is quantized pre-RoPE or post-RoPE. In the former case, the quantized key cache lacks positional information during decoding, which can produce incorrect outputs if

²Through the rest of this paper, we refer a $n \times k \times l$ tile as the dot product between $n \times k$ activations and $k \times l$ weights

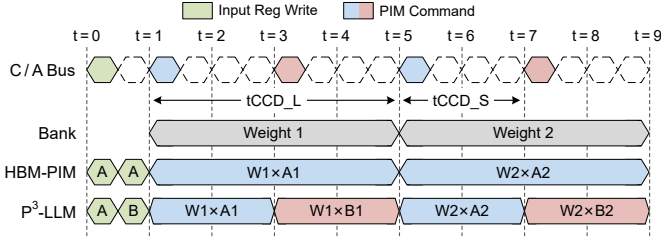


Fig. 7: Command timing of HBM-PIM and P³-LLM.

multiplied directly with the query. Therefore, an online RoPE operation must be applied to the key cache in each decoding iteration. Fortunately, RoPE is essentially a few element-wise operations with negligible runtime overhead [93]. Since RoPE is performed on NPU and its outputs are high-precision, we opt to calculate the following $Q \cdot K^T$ on NPU if the key cache is quantized before RoPE. In contrast, the post-RoPE quantized key cache can be directly multiplied with the query, which permits $Q \cdot K^T$ to be computed by the low-precision PCU.

C. Optimization on Quantized Computation Flow

Quantization Granularity. The quantization granularity has a direct impact on the computation flow. For query and KV-cache, we employ per-head quantization, where a group of D_h elements are quantized together and D_h is the head size. This allows immediate quantization of each newly generated token and eliminates the need to buffer a window of FP16 KV-cache as done in prior works [29], [61]. Additionally, weights are quantized per-group with a group size of 128 following BitMoD [6]. Activations are quantized per-token using FP8-E4M3. Finally, attention-score quantization does not need scaling since FP8-S0E4M4 already covers the required numerical range. Instead, we directly round the least significant bits of every FP16 attention-score and keep its highest 4 mantissa bits.

Quantized Computation Flow. As shown in Fig. 6(c), P³-LLM performs quantized matrix multiplication on the low-precision PCU, and implements operator fusion to minimize the runtime overhead of dequantization, which needs to be executed on NPU using high-precision arithmetic. For linear layers, the dequantization scaling is performed after matrix multiplication. For $Q \cdot K^T$, since the post-RoPE key cache contains per-channel smoothing factors (S_{SF}), we fuse S_{SF} into query via element-wise multiplication prior to FP8 quantization. Similarly, for $P \cdot V$, we fuse the per-value-head scaling factors (S_V) into attention-scores before FP8 quantization. One potential issue of fusing S_V is that it may cause the resulting attention-scores' magnitude to become larger than one, which is outside the range of FP8-S0E4M4. To prevent this, we apply a second-level scaling that divides S_V with its maximum $S_{V_{max}}$, resulting in $S_V \in [0, 1]$. Note that the quantization scaling factor is always positive by definition in Eq. 1, so fusing S_V still produces unsigned attention-scores. This second-level scaling factor $S_{V_{max}}$ is later multiplied back into the result of $P \cdot V$.

D. Throughput-Enhanced PCU

Despite the proposed low-precision PCU achieving $4\times$ the computational throughput than HBM-PIM, it still supports

only GEMV operations. To further exploit data reuse opportunities inside the PCU, one can increase the activation tile size from 1×4 to $n \times 4$ (where $n > 1$), and compute a $n \times 4 \times 16$ GEMM tile. But this will increase the number of multipliers by $n\times$, leading to considerable area overhead. A direct workaround is also to reduce the weight size by $n\times$, resulting in a $n \times 4 \times (16/n)$ GEMM tile without the need for additional multipliers. Nevertheless, this approach causes severe under-utilization of the PIM bandwidth because of the inherent 256-bit column access granularity of HBM.

Instead of adopting spatial data, we introduce a novel optimization that exploits temporal data reuse inside the PCU. Our key insight is that the proposed low-precision PCU not only reduces the area, but also permits a higher operating frequency compared to the baseline HBM-PIM [49]. Specifically, our evaluation (detailed in Section VI-A) demonstrates that the PCU of P³-LLM can reliably achieve $2\times$ higher frequency than that of HBM-PIM using FP32 accumulators. Given that HBM-PIM operates at a clock cycle of t_{CCD_L} , which corresponds to 4 memory bus cycles, the proposed PCU can instead operate P³-LLM at t_{CCD_S} , which is typically a half of t_{CCD_L} across all HBM generations [33], [36], [37].

Fig. 7 compares the command timing between HBM-PIM and P³-LLM. Initially, the host NPU sends input activations to the PCU register. Subsequently, a PIM command is initiated every t_{CCD_L} on HBM-PIM, which reads out 256-bit weights from the DRAM row buffer and performs a GEMV operation. P³-LLM allows the same weight slice to be reused twice by two different inputs, effectively doubling the throughput for cases where multiple inputs share the same weight. Our simulation shows that the PIM power of P³-LLM increases by 28% when its PCU operates at t_{CCD_S} instead of t_{CCD_L} , resulting in $1.56\times$ better energy efficiency. This is because the majority of PIM power dissipation comes from DRAM cell access and column decoder switching, which do not change as the throughput-enhanced PCU reuses the same set of weights from the row buffer across two consecutive t_{CCD_S} windows.

VI. EVALUATION

A. Experimental Methodology

Models and Datasets. We select eight representative models from Llama and Mistral families with varying sizes. Specifically, we evaluate Llama-1-(7B, 13B) [84]; Llama-2-(7B, 13B) [67]; Llama-3.1-8B [64]; Llama-3.2-3B [66]; and Mistral-7B-(v0.1, v0.3) [38]. We obtain the pre-trained models from their HuggingFace repository, and implement the proposed W4A8KV4P8 quantization algorithm in PyTorch. We apply pre-RoPE key-cache quantization to Llama-1 and Llama-2 given their short sequence length, while Llama-3 and Mistral adopt post-RoPE key-cache quantization. For accuracy evaluation of quantized models, we measure the perplexity³ on Wikitext-2 [63] and C4 [14] datasets. Additionally, we evaluate the accuracy on three difficult logical and mathematical reasoning tasks, including MMLU [26], ARC-Challenge [10], and GSM8K [74]. For these reasoning tasks, we use the

³For perplexity comparison with other baseline works, we use a context length of 2K to align with their reported results. Additionally, Llama-1 supports a maximum context length of 2K; exceeding this limit results in infinite perplexity across all baselines.

TABLE IV. Wikitext-2 and C4 perplexity ($\downarrow$) under different quantization methods and precisions. For both KV-cache-only and weight-activation quantization, we highlight the best perplexity results in bold.

Dataset	Method [†]	Precision	Llama-1		Llama-2		Llama-3		Mistral		Mean Δ ppl
			7B	13B	7B	13B	3.1-8B	3.2-3B	7B-v0.1	7B-v0.3	
Wikitext	Baseline	FP16	5.68	5.09	5.47	4.88	6.24	7.81	5.25	5.32	0
	Oaken	KV4	5.75	5.15	5.53	4.93	6.37	8.01	5.28	5.35	0.08
	P ³ -LLM	KV4	5.72	5.12	5.51	4.92	6.35	7.96	5.29	5.35	0.06
	QuaRot	W4A8KV4	5.92	5.28	5.70	5.07	6.80	8.48	5.41	5.49	0.30
	QoQ	W4A8KV4	5.89	5.24	5.69	5.07	6.79	8.56	5.42	5.50	0.30
	P ³ -LLM	W4A8KV4P8	5.81	5.22	5.65	5.01	6.75	8.39	5.41	5.48	0.25
C4	Baseline	FP16	7.08	6.61	6.97	6.47	8.96	10.43	7.74	7.83	0
	Oaken	KV4	7.15	6.66	7.04	6.52	9.10	10.69	7.79	7.87	0.09
	P ³ -LLM	KV4	7.11	6.63	7.01	6.50	9.09	10.62	7.78	7.87	0.07
	QuaRot	W4A8KV4	7.31	6.77	7.26	6.68	9.95	11.73	8.05	8.16	0.48
	QoQ	W4A8KV4	7.28	6.75	7.22	6.65	9.72	11.57	7.94	8.04	0.38
	P ³ -LLM	W4A8KV4P8	7.23	6.72	7.14	6.59	9.64	11.31	7.94	8.03	0.31

[†] For weight-activation quantization, QuaRot and QoQ maintain query and attention-scores in FP16. Whereas P³-LLM quantizes attention-scores to FP8-S0E4M4 across all models, and quantizes query to FP8-E4M3 for Llama-3 and Mistral.

instruction-tuned variants of Llama-3.1-8B and Llama-3.2-3B as suggested in the LM-Eval framework [19].

Algorithm Baselines. We compare P³-LLM with SoTA quantization algorithms, including Oaken [42], QuaRot [2], and QoQ [53]. Oaken is a 4-bit KV-cache quantization method that determines the key-cache outlier threshold offline using a calibration dataset. We compare Oaken with P³-LLM under KV-cache-only quantization for a fair comparison. QuaRot and QoQ implement aggressive W4A8KV4 quantization using the standard integer format. For all baseline algorithms, we use their official GitHub repository to reproduce the accuracy.

Hardware Implementation. We model P³-LLM’s PCU at RTL-level using SystemVerilog and synthesize it using Synopsys Design Compiler under TSMC 28nm technology. We scale the area and power results from our RTL simulation to HBM-PIM’s 20nm technology using DeepScaleTool [80]. We further scale the area overhead by taking into account the effects of DRAM process, which has 10 $\times$ lower transistor density than the logic process under the same feature size [13]. For performance evaluation, we develop a cycle-level simulator to model the P³-LLM system with 4 NPU cores and 16 pseudo HBM channels. The NPU design is based on [27], where each core contains a 128 $\times$ 128 systolic array, a 128-way vector processing unit, and 16MB on-chip scratchpad that is modeled with CACTI [5]. The PIM subsystem is simulated following the methodology of Newton [23], which is extended to model the computation flow of the proposed PCU. We set the operational frequency of NPU and PCU as 1 GHz and 500 MHz, respectively, considering t_{CCD_S} of HBM2 that is 2 DRAM clock cycles.

Accelerator Baselines. To evaluate hardware performance, we compare P³-LLM against three baselines: (1) An NPU accelerator running FP16 models, without PIM support in its DRAM; (2) An NPU accelerator integrated with HBM-PIM [49] running FP16 models; (3) A SoTA LLM accelerator, Ecco [8], employing W4A8KV4 quantization with k-means codebooks and Huffman encoding. All baseline systems share the same NPU and DRAM configurations as P³-LLM with comparable total area. For comparison of accelerator performance, we

focus on the decoding stage that typically dominates the end-to-end inference latency of edge applications [20], [52]. We use a context length of 4K that covers most edge scenarios as suggested in prior works [16], [50]. Notably, for accelerator evaluation, we only assess Llama-2-(7B, 13B), Llama-3.1-8B, Llama-3.2-3B, and Mistral-7B-v0.3, since the remaining models share the same size and/or architecture.

B. Model Performance

Perplexity Results. Table IV summarizes the model perplexity of different quantization methods. For KV-cache quantization, on average, P³-LLM outperforms Oaken and incurs only < 0.1 perplexity loss. Oaken also has a higher effective KV-cache precision of 4.8 bits due to its inefficient offline calibration, which keeps a large ratio ($\sim 10\%$) of outliers in high precision. On the other hand, P³-LLM employs per-head INT4-Asym quantization, where every 128 elements (i.e., the head dimension size of all evaluated models) share a 16-bit scaling factor and a 4-bit zero-point, resulting in an effective precision of 4.16 bits. Hence, P³-LLM achieves better perplexity even under a lower precision, demonstrating the effectiveness of our dynamic input-aware smoothing for mitigating outliers. For weight-activation quantization, on average, P³-LLM outperforms QuaRot and QoQ with 39% and 22% lower perplexity loss, respectively, despite performing more aggressive quantization with 8-bit queries and attention-scores. This highlights the strength of employing our operand-dependent, hybrid numerical formats in preserving model quality under aggressive quantization.

Accuracy Results. Table V shows the accuracy of different quantization methods. On average, P³-LLM achieves 1.04% higher accuracy than Oaken under 4-bit KV-cache quantization. Regarding weight-activation quantization, P³-LLM improves the average accuracy by a large margin of 2.57% and 3.05% compared to QuaRot and QoQ, respectively. Moreover, the accuracy improvement is particularly pronounced on Llama-3.2-3B. Both QuaRot and QoQ use integer formats that do not fit well to LLM tensor distributions, and rely on calibration datasets that lead to severe overfitting, particularly for smaller models with limited knowledge capacity.

TABLE V. Accuracy ($\uparrow$) of reasoning tasks under different quantization methods. Oaken and the top P³-LLM use KV4 quantization. QuaRot and QoQ use W4A8KV4 quantization, while the bottom P³-LLM uses W4A8KV4P8 quantization.

Model Dataset	Llama-3.1-8B			Llama-3.2-3B			Avg.
	MMLU	ARC-C	GSM8K	MMLU	ARC-C	GSM8K	
FP16	72.30	84.38	84.50	63.26	76.71	76.64	76.30
Oaken	67.31	83.70	81.80	59.93	76.79	75.66	74.20
P ³ -LLM	71.04	83.87	83.02	61.57	76.02	75.89	75.24
QuaRot	67.95	81.65	79.98	56.11	72.35	68.69	71.12
QoQ	67.46	81.31	79.53	54.11	72.18	69.22	70.64
P ³ -LLM	69.22	81.74	82.03	60.05	75.01	74.09	73.69

TABLE VI. Ablation study on the quantization techniques of P³-LLM (gray-shaded). The evaluation metric is Wikitext-2 perplexity ($\downarrow$).

Quantization Method	Llama-2-7B	Llama-3.1-8B
Baseline FP16	5.47	6.24
+ Pre-RoPE INT4 KV-cache quant	5.58 ($\uparrow$ 0.11)	6.52 ($\uparrow$ 0.28)
+ Post-RoPE INT4 KV-cache quant	5.61 ($\uparrow$ 0.03)	6.52 ($\cdot$)
→ Dynamic key-cache smoothing	5.51 ($\downarrow$ 0.10)	6.35 ($\downarrow$ 0.17)
+ INT4 weight quant	5.64 ($\uparrow$ 0.13)	6.79 ($\uparrow$ 0.44)
→ 4-bit BitMoD weight quant	5.63 ($\downarrow$ 0.01)	6.72 ($\downarrow$ 0.07)
+ FP8-E4M3 attn-score quant	5.67 ($\uparrow$ 0.04)	6.81 ($\uparrow$ 0.09)
→ FP8-S0E4M4 attn-score quant	5.63 ($\downarrow$ 0.04)	6.73 ($\downarrow$ 0.08)
+ INT8 activation quant	5.72 ($\uparrow$ 0.09)	6.84 ($\uparrow$ 0.11)
→ FP8-E4M3 activation quant	5.65 ($\downarrow$ 0.07)	6.75 ($\downarrow$ 0.09)

Algorithm Ablation Study. We conduct ablation studies on Llama-2-7B and Llama-3.1-8B to evaluate the accuracy gain of different quantization techniques used in P³-LLM. As shown in Table VI, we start with pre-RoPE and post-RoPE KV-cache quantization, and observe noticeable perplexity degradation. This is because both cases do not address the outlier issue of key cache. The proposed dynamic key-cache smoothing significantly improves the perplexity of Llama-2-7B and Llama-3.1-8B by 0.10 and 0.17, respectively. Then, adopting BitMoD for 4-bit weight quantization recovers some perplexity loss of INT4 weights. Subsequently, quantizing attention-scores to the proposed FP8-S0E4M4 format has negligible impact on perplexity. However, applying per-token activation quantization with INT8 brings another major perplexity degradation, which is finally recovered by the adopted FP8-E4M3 activation quantization. To summarize, P³-LLM explores optimal numerical formats that better adapt to different operands, thus minimizing their quantization error. In the presence of key-cache outliers, P³-LLM employs a novel smoothing technique to mitigate their impact.

Analysis of Key-cache Quantization Error. Fig. 8 presents the normalized key-cache quantization error of Oaken, QoQ, and P³-LLM on the Wikitext-2 and C4 datasets across all layers of Llama-2-7B. For each layer, we normalize the error to the average magnitude of all samples in the dataset. Oaken and QoQ rely on Wikitext-2 and Pile [18] as calibration datasets to address the key-cache outliers. Although Oaken has similar quantization error as P³-LLM on Wikitext-2, its error becomes more pronounced on C4 due to overfitting. QoQ has the highest error since the calibration statistics obtained from Pile does not generalize well to both Wikitext-2 and C4. In contrast, the dynamic input-aware smoothing of P³-LLM yields the lowest quantization error, as it does not overfit to any calibration dataset.

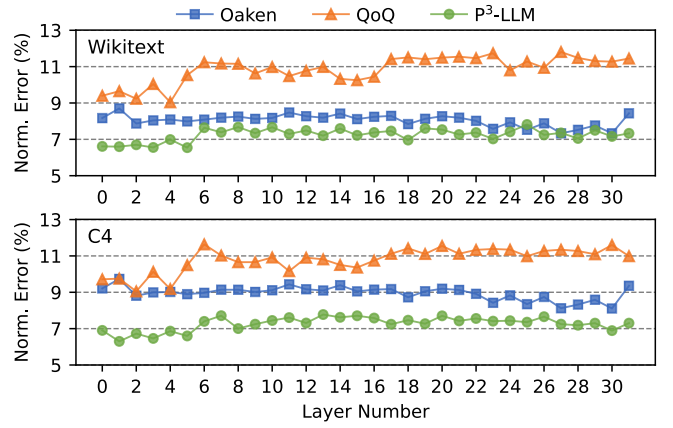


Fig. 8: Normalized layer-wise key-cache quantization error of Llama-2-7B on Wikitext-2 and C4 datasets. Oaken and QoQ use Wikitext-2 and Pile as calibration datasets, respectively.

C. Hardware Performance

Speedup. Fig. 9 depicts the normalized decoding speed of P³-LLM against three baseline accelerators with batch sizes varying from 1 to 8. The HBM-PIM system outperforms NPU at low batch sizes of 1 and 2 across all models, primarily due to its higher internal bandwidth. However, as the batch size reaches 4, the performance advantage of HBM-PIM gradually diminishes for Llama-2 and even disappears for Llama-3 and Mistral. This is because the linear layer exhibits more data reuse with increasing batch sizes, which NPU can exploit to boost performance. Additionally, the GQA mechanism of Llama-3 and Mistral offer inherent data reuse opportunities that HBM-PIM fail to exploit. Furthermore, both NPU and HBM-PIM deliver lower performance than Ecco that leverages quantization to reduce the demand of memory bandwidth. On the other hand, P³-LLM offers substantial performance gains over all baseline accelerators, yielding average speedups of 7.8 $\times$, 4.9 $\times$, and 2.0 $\times$ over NPU, HBM-PIM, and Ecco, respectively. These performance gains of P³-LLM stem from its careful algorithm-hardware co-design of mixed-precision quantization and efficient PCU architecture. Interestingly, P³-LLM demonstrates its highest speedup at a batch size of 2, owing to its throughput-enhanced PCU that allows processing two input vectors within the same $t_{\text{CCD,L}}$ window.

Energy Consumption. Fig. 10 depicts the breakdown of energy consumption for attention and linear layers across different accelerators and batch sizes. On average, P³-LLM yields 6.3 $\times$, 3.5 $\times$, and 2.1 $\times$ better energy efficiency over NPU, HBM-PIM, and Ecco, respectively. These energy savings are attributed to the reduced memory footprint offered by the W4A8KV4P8 quantization scheme, as well as a PIM architecture co-design that allows most layers to be accelerated by the low-precision PCU. As the batch size increases, NPU can take advantage of data reuse by loading the model weights only once and processing all input requests within a batch simultaneously. In contrast, the PIM accelerator of HBM-PIM only supports GEMV, requiring the model weights to be repetitively fetched from the DRAM bank for processing each input request. Thus, the energy consumption of linear layers in HBM-PIM increases significantly with larger batch sizes. Compared to HBM-PIM, the throughput-enhanced PCU in

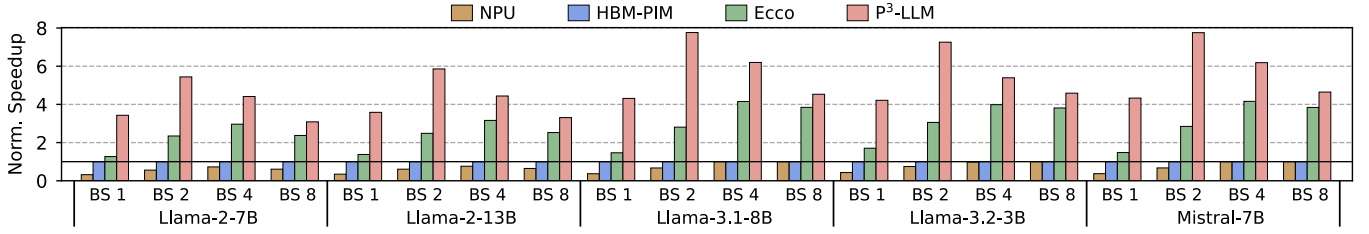


Fig. 9: Normalized speedup (↑) vs. batch size (BS) for different accelerator systems. The context length is 4K.

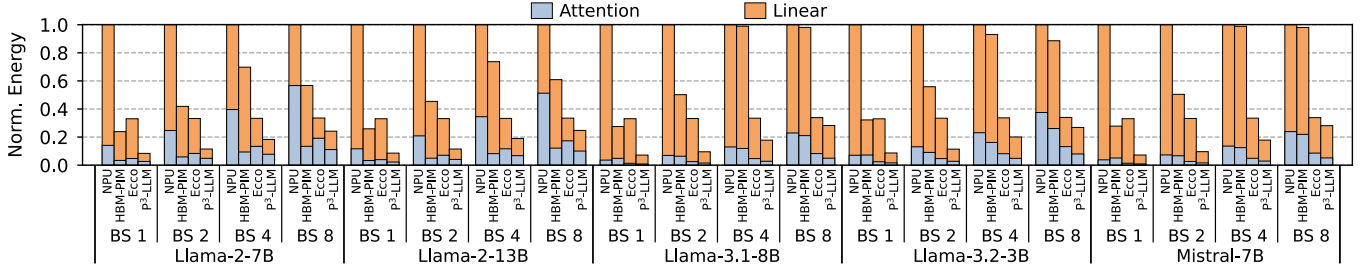


Fig. 10: Normalized energy consumption (↓) vs. batch size (BS) and the breakdown for attention and linear layers. The context length is 4K.

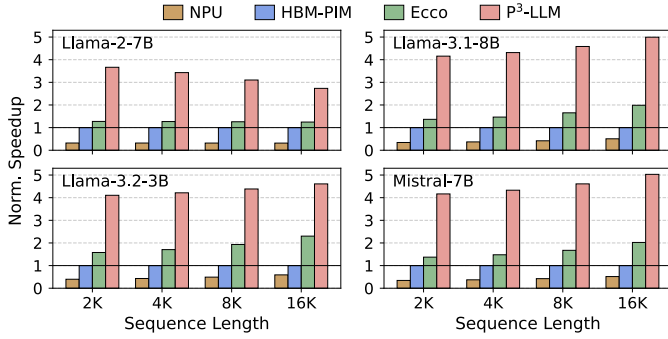


Fig. 11: Normalized decoding speedup (↑) across different context lengths under single-batch inference.

P³-LLM enables each memory access to be reused twice, effectively reducing the overhead of duplicated DRAM row activations and the associated energy consumption.

Sensitivity to Context Length. Single-batch inference remains one of the most important scenarios for edge LLM deployment. We analyze the impact of context length on the single-batch decoding performance of different accelerator systems. As shown in Fig. 11, scaling the context length from 2K to 16K yields additional performance gains for P³-LLM across all models, except Llama-2-7B. With a longer context length, the attention layer starts to dominate the overall runtime due to the increased KV-cache footprint. P³-LLM effectively mitigates this overhead through its 4-bit KV-cache quantization and attention offloading to the mixed-precision PIM accelerator. For Llama-2-7B, since P³-LLM applies key-cache quantization before RoPE, the $Q \cdot K^T$ computation is offloaded to NPU, resulting in slightly lower speedup under longer context length.

Comparison with Existing Low-Precision PIM. We compare the performance of P³-LLM with Pimba [44], a SoTA low-precision PIM architecture adopting the 8-bit microscaling format [79]. The original Pimba employs KV-cache-only quantization since it targets cloud serving scenarios, where KV-cache

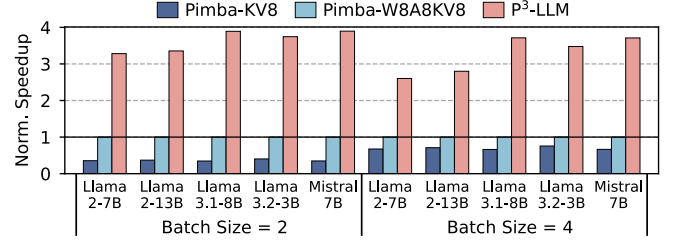


Fig. 12: Normalized decoding speedup (↑) of Pimba and P³-LLM. The context length is 4K.

dominates the memory footprint. Hence, we also examine an enhanced version of Pimba with 8-bit weight-activation quantization. Fig. 12 presents the normalized decoding speed of P³-LLM and Pimba under batch sizes of 2 and 4. The original Pimba has the lowest performance because at small batch sizes, weights can dominate the overall memory footprint but remain unquantized. By adopting 8-bit weight-activation quantization, on average, the enhanced Pimba achieves $2.1\times$ better performance compared to its original design. The proposed P³-LLM further yields an average of $3.4\times$ performance boost compared to the enhanced Pimba. By quantizing weights and KV-cache to 4 bits with minimal accuracy loss, P³-LLM reduces the memory access compared to Pimba. Moreover, the throughput-enhanced PCU of P³-LLM enables temporal input reuse to double the computational throughput, facilitating efficient execution of low-batch linear layers and GQA.

Comparison with Software Quantization. We compare P³-LLM with two SoTA software quantization algorithms, SmoothQuant [88] and AWQ [52] running on our baseline NPU. Fig. 13 illustrates the decoding throughput of different methods across various batch sizes from 1 to 8. On average, P³-LLM yields $3.9\times$ and $3.0\times$ higher throughput than SmoothQuant and AWQ, respectively. The performance gains of P³-LLM arise from its carefully co-designed W4A8KV4P8 quantization algorithm and low-precision PIM architecture. Compared to P³-LLM, SmoothQuant and AWQ adopt more conservative 8-bit weight-activation quantization and 4-bit

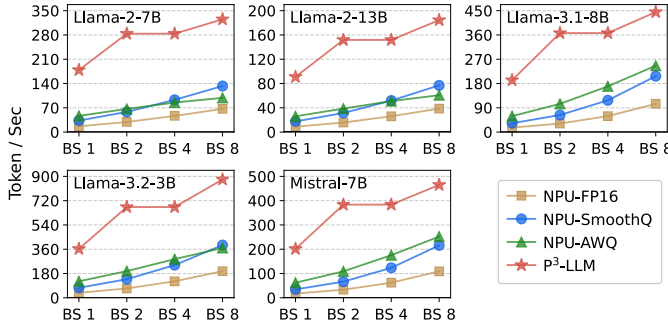


Fig. 13: Decoding throughput ($\uparrow$) vs. batch size (BS) of different quantization methods. The context length is 4K.

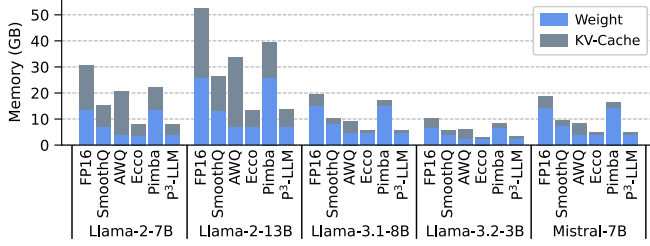


Fig. 14: Memory consumption ($\downarrow$) of different quantization methods during decoding. The batch size is 8 and the context length is 4K.

weight-only quantization, respectively. Thus, both methods provide moderate reductions in memory traffic during decoding, which is further exacerbated by the limited off-chip DRAM bandwidth.

Memory Analysis. We examine the memory consumption of weights and KV-cache across various quantization methods during the decoding phase. As shown in Fig. 14, Ecco and P³-LLM achieve substantial memory reductions of 3.8 $\times$ and 3.7 $\times$ over the FP16 baseline, respectively, owing to their aggressive 4-bit quantization of weights and KV-cache that dominate the memory access during LLM decoding. Compared to P³-LLM, Ecco has a slightly smaller memory footprint due to its compact Hoffman encoding of quantization codebooks and metadata. However, this advantage is largely negated by the constrained bandwidth between NPU and DRAM, leading to much lower decoding speed than P³-LLM.

Area and Power. Following established methodologies [44], [94], we quantify the total area overhead of P³-LLM and compare to that of HBM-PIM. Note that the original HBM-PIM described in [49] supports diverse operations such as element-wise multiplication and addition. Given that LLM decoding is dominated by MAC operations, we only model necessary MAC hardware in HBM-PIM, eliminating complicated instruction decoder and unnecessary operand registers. As shown in Table VII, P³-LLM introduces a total HBM area overhead of 17.5%, well below the 25% maximum logic ratio recommended by prior works [23], [49]. Compared to HBM-PIM using FP16 arithmetic, P³-LLM incurs only 1.1% larger area, yet this increase is justified by delivering an average of 4.9 $\times$ speedup for edge LLM inference, while maintaining usable accuracy.

We also compare the P³-LLM PE with two mixed-precision PEs of SoTA LLM accelerators, BitMoD [6] and MANT [29].

TABLE VII. Comparison between HBM-PIM and P³-LLM.

	PCU Area [mm ²]		HBM Area	Avg. Norm.
	Compute	Buffer	Overhead ($\downarrow$)	Speedup ($\uparrow$)
HBM-PIM	7.7	6.2	16.4%	1.0 $\times$
P ³ -LLM (Ours)	8.4	6.2	17.5%	4.9 $\times$

TABLE VIII. The area and energy consumption of different PE design under 1GHz. The numbers are normalized to that of an FP16 MAC.

Type	MAC/Cycle	Area [μ m ²]	Energy [pJ/MAC]
HBM-PIM	1 MAC	1023.1 (1.00 $\times$)	0.69 (1.00 $\times$)
MANT	2 MACs [‡]	717.3 (0.70 $\times$)	0.40 (0.58 $\times$)
BitMoD	2 MACs [‡]	1291.6 (1.26 $\times$)	0.61 (0.88 $\times$)
P ³ -LLM (Ours)	4 MACs [‡]	1109.2 (1.08 $\times$)	0.18 (0.26 $\times$)

[‡] Normalized to MACs/Cycle under 4-bit weight quantization.

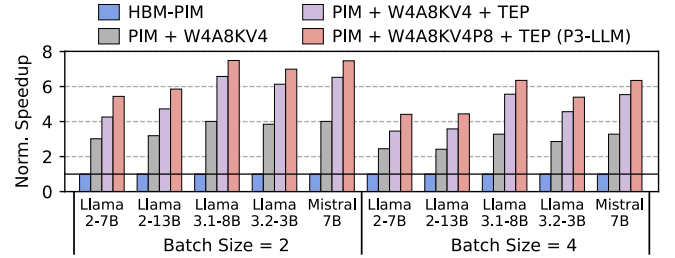


Fig. 15: Ablation study on different architectural techniques of P³-LLM. The context length is 4K.

Table VIII shows the area and power of different PE designs under 1GHz frequency. BitMoD exhibits the lowest hardware efficiency, as it requires an expensive FP32 accumulator to handle unquantized activations. While MANT employs weight-activation quantization, its adaptive numerical type decomposes the weight-activation multiplication into two partial sums with high bit-width. This necessitates an expensive adder to add the two partial sums before accumulation, resulting in large area and energy overhead. P³-LLM delivers superior performance over the FP16 PE of HBM-PIM, with 3.8 $\times$ higher energy efficiency per MAC. This substantial efficiency gain is attributed to our efficient quantization approach that reduces the bit-width for both operands and intermediate computation.

Architecture Ablation Study. We conduct ablation studies to assess the performance gain of different architectural techniques proposed by P³-LLM. Four designs are evaluated: (1) The baseline HBM-PIM accelerator running FP16 models; (2) A PIM accelerator supporting W4A8KV4 quantized models; (3) A PIM accelerator incorporating our throughput-enhanced PCU (TEP) to accelerate W4A8KV4 models; (4) The proposed P³-LLM with 8-bit attention-score on top of W4A8KV4 and throughput-enhanced PCU. Fig. 15 illustrates the normalized performance under batch sizes of 2 and 4. On average, W4A8KV4 quantization achieves 3.3 $\times$ speedup over HBM-PIM, and adopting the throughput-enhanced PCU to exploit data reuse offers an additional 1.6 $\times$ speedup. Finally, with 8-bit attention-score quantization, on average, P³-LLM achieves another 1.2 $\times$ performance gain by enabling the low-precision PCU to fully accelerate the self-attention module.

Applicability to Large-Batch Decoding. Although P³-LLM

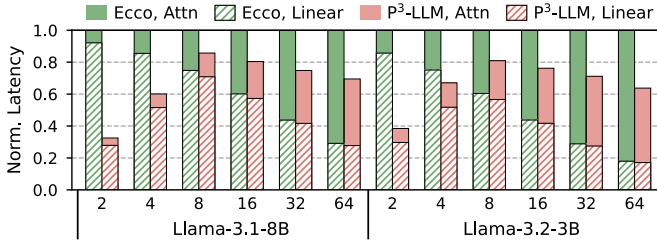


Fig. 16: Normalized decoding latency ($\downarrow$) of Ecco and P³-LLM across a wide range of batch sizes from 2 to 64. The context length is 4K.

mainly targets low-batch edge inference, it is also applicable to large-batch serving. To demonstrate this applicability, Fig. 16 presents the decoding latency breakdown of Ecco and P³-LLM for two Llama-3 models across a wide range of batch sizes from 2 to 64. Notably, Ecco and P³-LLM have similar latency for linear layers when the batch size reaches 8, as the PIM hardware becomes compute-bound. To address this, P³-LLM offloads linear layers to NPU for more efficient execution. Interestingly, as the batch size continues to grow, P³-LLM regains its performance advantage, which is attributed to the increasing dominance of attention layers in the overall runtime. Specifically, the GQA of Llama-3.1-8B and Llama-3.2-3B has a group size of 4 and 3, respectively, which still exhibits low data reuse opportunities. Thus, P³-LLM can leverage its high internal PIM bandwidth and throughput-enhanced PCU to accelerate the fully quantized attention module, resulting in better performance than Ecco.

VII. DISCUSSION

Broader Applicability of P³-LLM. Apart from HBM-PIM, the proposed low-precision PCU of P³-LLM is generally applicable to other DRAM variants such as GDDR [34] and LPDDR [35]. For instance, the LPDDR-PIM [41] introduced by Samsung adopts a similar PCU design as HBM-PIM, both containing a 16-way SIMD FP16 MAC unit that receives 256-bit inputs and weights to compute GEMV. Furthermore, the throughput-enhanced PCU in P³-LLM can also be applied to LPDDR, whose t_{CCD_S} is a half of t_{CCD_L} .

VIII. RELATED WORKS

Integer-based LLM Quantization. Numerous studies have explored quantization algorithms based on conventional integer formats to reduce the memory footprint and computational demand of LLMs [2], [12], [17], [28], [31], [42], [43], [46], [52]–[55], [57]–[59], [61], [70], [82], [88], [96]. While weights are easy to quantize using integer formats, activations usually exhibit much larger dynamic range with significant quantization error. To address this, prior works employ various techniques, such as maintaining outliers at higher precision [12], [42], [96], smoothing activation channels [53], [88], and applying matrix rotation to activations [2], [43], [59]. However, these methods require either offline calibration with potential overfitting, or considerable online computation overhead. Our work distinguishes itself by seamlessly blending lightweight dynamic smoothing and hybrid numerical formats to achieve a balanced trade-off between model accuracy and hardware efficiency.

Custom Quantization Formats. Recently, there has been significant research on custom numerical formats for quantization in the architecture community [6]–[9], [15], [16], [21], [22], [29], [32], [47], [56], [75], [76], [78], [79], [87], [89], [91]. Although these proposals achieve good hardware efficiency, they struggle to outperform existing algorithmic solutions under the aggressive W4A8KV4 configuration [2], [53]. On the other hand, P³-LLM introduces a novel operand-dependent quantization scheme using hybrid numerical formats, achieving better accuracy than SoTA algorithmic solutions. In addition to academic proposals, custom quantization formats have been widely adopted by industry. The microscaling format [79], standardized by Open Compute Project, applies a shared 8-bit exponent to a group of low-precision operands. Both NVIDIA and AMD add support for FP4 and FP8 formats in their flagship GPUs [1], [72]. The latest NVIDIA Blackwell architecture introduces the NVFP4 format [71], which enhances the standard FP4 data type with a two-level scaling strategy.

PIM-based LLM Inference Systems. There is an abundance of works on LLM inference using NPU-PIM heterogeneous accelerators [24], [25], [27], [30], [40], [41], [44], [45], [51], [62], [77], [81], [94]. Samsung and SK Hynix have commercialized PIM products based on DDR and HBM technology [40], [41]. AttAcc [77], IANUS [81], and NeuPIMs [27] explore operator mapping and scheduling to improve the utilization of NPU and PIM during inference. These works focus exclusively on FP16 LLM inference, making them orthogonal to our quantization method. Pimba [44], PLAIN [30], and LP-Spec [24] design low-precision arithmetic units to enhance the throughput of PIM. However, they mainly target conservative quantization configurations such as W8A8 and W6A6. In contrast, our work explores more aggressive W4A8KV4P8 quantization to achieve better trade-off between model accuracy, memory footprint, and computational efficiency.

IX. CONCLUSION

In this paper, we present P³-LLM, an algorithm-hardware co-design solution for efficient edge LLM inference on heterogeneous NPU-PIM accelerators. On the algorithm side, P³-LLM develops a novel quantization scheme using hybrid numerical formats to quantize all major LLM operands, while maintaining superior model accuracy. On the hardware side, P³-LLM is equipped with optimized low-precision dataflow and an area-efficient PCU architecture to support the proposed quantization scheme. Our evaluation demonstrates that the P³-LLM algorithm outperforms SoTA KV-cache-only and weight-activation quantization methods: Oaken, QuaRot, and QoQ. Compared to SoTA LLM accelerators: HBM-PIM, Ecco, and Pimba, the P³-LLM accelerator yields 4.9 $\times$, 2.0 $\times$, and 3.4 $\times$ speedups, respectively.

X. ACKNOWLEDGMENT

We thank the anonymous reviewers of HPCA-2025 and ISCA-2026 for their helpful and constructive feedback. This work was supported in part by the National Science Foundation under Grant No. 2339084, the European Research Council (ERC) under grant agreement No. 101088865, the Flanders AI Research Program, and long-term structural Methusalem funding by the Flemish Government.

REFERENCES

- [1] AMD, “AMD INSTINCT™ MI350X GPU.” [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/product-briefs/amd-instinct-mi350x-gpu-brochure.pdf>
- [2] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, “QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs,” *Advances in neural information processing systems (NeurIPS)*, 2024.
- [3] J. Bai, S. Bai, S. Yang, S. Wang, S. Tan, P. Wang, J. Lin, C. Zhou, and J. Zhou, “Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond,” *arXiv preprint arXiv:2308.12966*, 2023.
- [4] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding,” *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [5] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “CACTI 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, June 2017.
- [6] Y. Chen, A. F. AbouElhamayed, X. Dai, Y. Wang, M. Andronic, G. A. Constantinides, and M. S. Abdelfattah, “BitMoD: Bit-serial Mixture-of-Datatype LLM Acceleration,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
- [7] Y. Chen, X. Dai, J. Hyun, C.-C. Chang, W. Jang, Y. Wu, T. Tambe, J. sun Seo, and M. S. Abdelfattah, “RaZeR: Pushing the Limits of NVFP4 Quantization with Redundant Zero Remapping,” *arXiv preprint arXiv:2501.04052*, 2026.
- [8] F. Cheng, C. Guo, C. Wei, J. Zhang, C. Zhou, E. Hanson, J. Zhang, X. Liu, H. H. Li, and Y. Chen, “Ecco: Improving Memory Bandwidth and Capacity for LLMs via Entropy-Aware Cache Compression,” *ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025.
- [9] J. Chhugani, G. Jeong, B.-Y. Su, Y. Pan, H. Yang, A. Ankit, J. Yu, S. Deng, Y. Chen, N. Satish, and C. Kim, “Unveiling the Potential of Quantization with MXFP4: Strategies for Quantization Error Reduction,” *arXiv preprint arXiv:2603.08713*, 2026.
- [10] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have Solved Question Answering? Try ARC, the A12 Reasoning Challenge,” *arXiv preprint arXiv:1803.05457*, 2018.
- [11] DeepSeek AI, “DeepSeek R1.” [Online]. Available: <https://github.com/deepseek-ai/DeepSeek-R1>
- [12] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” *arXiv preprint arXiv:2208.07339*, 2022.
- [13] F. Devaux, “The true Processing In Memory accelerator,” *IEEE Hot Chips 31 Symposium (HCS)*, 2019.
- [14] J. Dodge, A. Marasovic, G. Ilharco, D. Groeneveld, M. Mitchell, and M. Gardner, “Documenting large webtext corpora: A case study on the colossal clean crawled corpus,” in *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [15] J. Dotzel, Y. Chen, B. Kotb, S. Prasad, G. Wu, S. Li, M. S. Abdelfattah, and Z. Zhang, “Learning from Students: Applying t-Distributions to Explore Accurate and Efficient Formats for LLMs,” *International Conference on Machine Learning (ICML)*, 2024.
- [16] C. Fang, M. Shi, R. Geens, A. Symons, Z. Wang, and M. Verhelst, “Anda: Unlocking Efficient LLM Inference with a Variable-Length Grouped Activation Data Format,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
- [17] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, “GPTQ: Accurate Post-training Compression for Generative Pretrained Transformers,” *International Conference on Learning Representations (ICLR)*, 2023.
- [18] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, “The Pile: An 800GB Dataset of Diverse Text for Language Modeling,” *arXiv preprint arXiv:2101.00027*, 2020.
- [19] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonnell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “The Language Model Evaluation Harness,” 2024. [Online]. Available: <https://zenodo.org/records/12608602>
- [20] R. Geens, M. Shi, A. Symons, C. Fang, and M. Verhelst, “Energy Cost Modelling for Optimizing Large Language Model Inference on Hardware Accelerators,” in *IEEE 37th International System-on-Chip Conference (SOCC)*, 2024.
- [21] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, F. Yang, Y.-B. Liu, M. Guo, and Y. Zhu, “OliVe: Accelerating Large Language Models via Hardware-friendly Outlier-Victim Pair Quantization,” *ACM/IEEE 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
- [22] C. Guo, C. Zhang, J. Leng, Z. Liu, F. Yang, Y.-B. Liu, M. Guo, and Y. Zhu, “ANT: Exploiting Adaptive Numerical Data Type for Low-bit Deep Neural Network Quantization,” *IEEE/ACM 55th Annual International Symposium on Microarchitecture (MICRO)*, 2022.
- [23] M. He, C. Song, I. Kim, C. Jeong, S. Kim, I. Park, M. Thottethodi, and T. N. Vijaykumar, “Newton: A DRAM-maker’s Accelerator-in-Memory (AiM) Architecture for Machine Learning,” *IEEE/ACM 53rd International Symposium on Microarchitecture (MICRO)*, 2020.
- [24] S. He, Z. Zhu, Y. He, and T. Jia, “LP-Spec: Leveraging LPDDR PIM for Efficient LLM Mobile Speculative Inference with Architecture-Dataflow Co-Optimization,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025.
- [25] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, “PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System,” *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.
- [26] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring Massive Multitask Language Understanding,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [27] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. T. Kim, D. Mahajan, and J. Park, “NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing,” *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [28] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization,” *Advances in neural information processing systems (NeurIPS)*, 2024.
- [29] W. Hu, H. Zhang, C. Guo, Y. Feng, R. Guan, Z. Hua, Z. Liu, Y. Guan, M. Guo, and J. Leng, “M-ANT: Efficient Low-bit Group Quantization for LLMs via Mathematically Adaptive Numerical Type,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
- [30] Y. Hu, F. Liu, Z. Wang, Y. Zhao, T. Yang, L. Jiang, and H. Guan, “PLAIN: Leveraging High Internal Bandwidth in PIM for Accelerating Large Language Model Inference via Mixed-Precision Quantization,” in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025.
- [31] J. Jang, Y. Kim, J. Lee, and J.-J. Kim, “FIGNA: Integer unit-based accelerator design for fp-int gemm preserving numerical accuracy,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.
- [32] W. Jang and T. Tambe, “BlockDialect: Block-wise Fine-grained Mixed Format Quantization for Energy-Efficient LLM Inference,” *International Conference on Machine Learning (ICML)*, 2025.
- [33] JEDEC, “High Bandwidth Memory (HBM3) DRAM,” 2021. [Online]. Available: <https://www.jedec.org/standards-documents/docs/jesd235a>
- [34] JEDEC, “Graphics Double Data Rate (GDDR6) SGRAM Standard,” 2023. [Online]. Available: <https://www.jedec.org/standards-documents/docs/jesd250d>
- [35] JEDEC, “Low Power Double Data Rate 5 (LPDDR5),” 2023. [Online]. Available: <https://www.jedec.org/standards-documents/docs/jesd209-5c>
- [36] JEDEC, “High Bandwidth Memory (HBM3) DRAM,” 2025. [Online]. Available: <https://www.jedec.org/standards-documents/docs/jesd238b01>
- [37] JEDEC, “High Bandwidth Memory (HBM4) DRAM,” 2025. [Online]. Available: <https://www.jedec.org/standards-documents/docs/jesd270-4>
- [38] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023.
- [39] N. P. Jouppi, D. H. Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. C. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. A. Patterson, “Ten Lessons From Three Generations Shaped Google’s TPuv4i: Industrial Product,” *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
- [40] G. Kim, J. Kim, N. Y. Kim, W. Shin, J.-H. Won, H. Joo, H. Choi, B. An, G. Shin, D. Yun, J. Kim, C. Kim, I.-H. Kim, J. Park, Y. Song, B. Yang, H. Lee, S. Park, W. Lee, S. Kim, Y. Park, Y. Jung, G.-H. Park, and E. Lim, “SK Hynix AI-Specific Computing Memory Solution: From

- AiM Device to Heterogeneous AiMX-xPU System for Comprehensive LLM Inference,” *IEEE Hot Chips 36 Symposium (HCS)*, 2024.
- [41] J. H. Kim, Y. Ro, J. So, S. Lee, S. Kang, Y. Cho, H. Kim, B. Kim, K. Kim, S.-S. Park, J.-S. Kim, S. Cha, W.-J. Lee, J. Jung, J. Lee, J. Lee, J. Song, S. Lee, J. Cho, J. Yu, and K. Sohn, “Samsung PIM/PNM for Transformer Based AI : Energy Efficiency on PIM/PNM Cluster,” *IEEE Hot Chips 35 Symposium (HCS)*, 2023.
 - [42] M. Kim, S. Hong, R. Ko, S. Choi, H. Lee, J. Kim, J.-Y. Kim, and J. Park, “Oaken: Fast and Efficient LLM Serving with Online-Offline Hybrid KV Cache Quantization,” *ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025.
 - [43] S. Kim, Y. Choi, B. Kim, J. Oh, and H.-J. Yoo, “GyRot: Leveraging Hidden Synergy between Rotation and Fine-grained Group Quantization for Low-bit LLM Inference,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026.
 - [44] W. Kim, Y. Lee, Y. Kim, J. Hwang, S. Oh, J. Jung, A. Huseynov, W. G. Park, C. H. Park, D. Mahajan, and J. Park, “Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving,” *IEEE/ACM 58th International Symposium on Microarchitecture (MICRO)*, 2025.
 - [45] H. Kwon, K. Koo, J. Kim, W. Lee, M. Lee, G. Jung, H. Lee, Y. Jung, J. Park, Y. Song, B. Yang, H. Choi, G. Kim, J. Won, W. Shin, C. Kim, S. Gyeongcheol, Y. Kwon, I. Kim, E. Lim, J. Kim, and J. Choi, “PIM-phony: Overcoming Bandwidth and Capacity Inefficiency in PIM-based Long-Context LLM Inference System,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026.
 - [46] J. Lee, W. Lee, and J. Sim, “Tender: Accelerating Large Language Models via Tensor Decomposition and Runtime Requantization,” *ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
 - [47] J. Lee, J. Park, S. Cha, J. Cho, and J. Sim, “MX+: Pushing the Limits of Microscaling Formats for Efficient Large Language Model Serving,” *IEEE/ACM 58th Annual International Symposium on Microarchitecture (MICRO)*, 2025.
 - [48] S. J. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Y. Ka, K.-D. Hwang, J.-J. Park, K. Kang, J. Kim, J. Jeon, N. Y. Kim, Y. Kwon, K. Vladimir, W. Shin, J.-H. Won, M. Lee, H. Joo, H. Choi, J. Lee, D.-Y. Ko, Y. Jun, K. yeong Cho, I. Kim, C. Song, C. Jeong, D.-H. Kwon, J. Jang, I. Park, J. H. Chun, and J. Cho, “A 1nm 1.25V 8Gb 16Gb/s/Pin GDDR6-Based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep Learning Application,” *IEEE Journal of Solid-State Circuits (JSSC)*, vol. 58, pp. 291–302, 2023.
 - [49] S. Lee, S. Kang, J. Lee, H. Kim, E. Lee, S. young Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, S. O, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, “Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product,” *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
 - [50] C. Li, Y. Yin, X. Wu, J. Zhu, Z. Gao, D. Niu, Q. Wu, X. Si, Y. Xie, C. Zhang, and G. Sun, “H2-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference,” *ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025.
 - [51] S. Li, Y. Chen, C. Li, Y. Fu, Z. Wang, Z. Yu, H. You, Z. Ye, W. Zhou, Y. Zhang, and Y. C. Lin, “ORCHES: Orchestrated Test-Time-Compute-based LLM Reasoning on Collaborative GPU-PIM Heterogeneous System,” *IEEE/ACM 58th Annual International Symposium on Microarchitecture (MICRO)*, 2025.
 - [52] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
 - [53] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, “QServe: W4A8KV4 Quantization and System Co-design for Efficient LLM Serving,” in *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
 - [54] F. Liu, J. Wang, N. Yang, Z. Wang, J. Zhao, L. Jiang, and H. Guan, “ASTER: Adaptive Dynamic Layer-Skipping for Efficient Transformer Inference via Markov Decision Process,” *Proceedings of the 33rd ACM International Conference on Multimedia*, 2025.
 - [55] F. Liu, Z. Wang, J. Xia, J. Zhao, J. Liu, H. Guan, and L. Jiang, “FlexQuant: A Flexible and Efficient Dynamic Precision Switching Framework for LLM Quantization,” *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
 - [56] F. Liu, N. Yang, H. Li, Z. Wang, Z. Song, S. Pei, and L. Jiang, “SPARK: Scalable and Precision-Aware Acceleration of Neural Networks via Efficient Encoding,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2024.
 - [57] F. Liu, N. Yang, Z. Wang, X. Zhu, H. Yao, X. Xiong, L. Jiang, and H. Guan, “BLOOM: Bit-Slice Framework for DNN Acceleration with Mixed-Precision,” *ACM/IEEE Design Automation Conference (DAC)*, 2025.
 - [58] F. Liu, N. Yang, J. Yang, Z. Wang, C. Guan, Y. Feng, L. Jiang, and H. Guan, “EARTH: An Efficient MoE Accelerator with Entropy-Aware Speculative Prefetch and Result Reuse,” *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2026.
 - [59] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, and T. Blankevoort, “SpinQuant: LLM quantization with learned rotations,” *International Conference on Learning Representations (ICLR)*, 2025.
 - [60] Z. Liu, C. Zhao, F. N. Iandola, C. Lai, Y. Tian, I. Fedorov, Y. Xiong, E. Chang, Y. Shi, R. Krishnamoorthi, L. Lai, and V. Chandra, “MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases,” in *International Conference on Machine Learning*, 2024.
 - [61] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache,” *International Conference on Machine Learning (ICML)*, 2024.
 - [62] K. Matsushima, Y. Okoshi, M. Motomura, and D. Fujiki, “AQPM: Breaking the PIM Capacity Wall for LLMs with In-Memory Activation Quantization,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2026.
 - [63] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *International Conference on Learning Representations (ICLR)*, 2017.
 - [64] Meta, “Introducing Llama 3.1: Our most capable models to date.” [Online]. Available: <https://ai.meta.com/blog/meta-llama-3-1/>
 - [65] Meta, “Llama-3.2-90B-Vision-Instruct.” [Online]. Available: <https://huggingface.co/meta-llama/Llama-3.2-90B-Vision-Instruct>
 - [66] Meta, “Llama 3.2: Revolutionizing edge AI and vision with open, customizable models.” [Online]. Available: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
 - [67] Meta, “Meta Llama 2.” [Online]. Available: <https://github.com/meta-llama/llama>
 - [68] Meta, “The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation.” [Online]. Available: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>
 - [69] P. Micikevicius, D. Stolic, N. Burgess, M. Cornea, P. K. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. F. Oberman, M. Shoenybi, M. Siu, and H. Wu, “FP8 Formats for Deep Learning,” *arXiv preprint arXiv:2209.05433*, 2022.
 - [70] Z. Mo, L. Wang, J. Wei, Z. Zeng, S. Cao, L. Ma, N. Jing, T. Cao, J. Xue, F. Yang, and M. Yang, “LUT Tensor Core: A Software-Hardware Co-Design for LUT-Based Low-Bit LLM Inference,” *ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2024.
 - [71] NVIDIA, “Introducing NVFP4 for Efficient and Accurate Low-Precision Inference.” [Online]. Available: <https://developer.nvidia.com/blog/introducing-nvfp4-for-efficient-and-accurate-low-precision-inference/>
 - [72] NVIDIA, “NVIDIA Blackwell GPU Architecture.” [Online]. Available: <https://resources.nvidia.com/en-us-blackwell-architecture/datasheet>
 - [73] Open AI, “Openai o3-mini.” [Online]. Available: <https://openai.com/index/openai-o3-mini/>
 - [74] OpenAI, “Gsm8k dataset.” [Online]. Available: <https://huggingface.co/datasets/openai/gsm8k>
 - [75] G. Park, H. Kwon, J. Kim, J. Bae, B. Park, D. Lee, and Y. Lee, “FIGLUT: An Energy-Efficient Accelerator Design for FP-INT GEMM Using Look-Up Tables,” *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
 - [76] G. Park, B. Park, M. Kim, S. Lee, J. Kim, B. Kwon, S. J. Kwon, B. Kim, Y. Lee, and D. Lee, “LUT-GEMM: Quantized Matrix Multiplication based on LUTs for Efficient Inference in Large-Scale Generative Language Models,” in *International Conference on Learning Representations (ICLR)*, 2024.
 - [77] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, “AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference,” *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
 - [78] A. Ramachandran, S. Kundu, and T. Krishna, “MicroScopiQ: Accelerating Foundational Models through Outlier-Aware Microscaling Quantization,” *ACM/IEEE 52nd Annual International Symposium on Computer Architecture (ISCA)*, 2025.
 - [79] B. D. Rouhani, R. Zhao, V. Elango, R. Shafipour, M. Hall, M. Mes-makhosroshahi, A. More, L. Melnick, M. Golub, G. Varatkar, L. Shao, G. Kolhe, D. Melts, J. Klar, R. L’Heureux, M. Perry, D. Burger, E. S.

- Chung, Z. Deng, S. Naghshineh, J. Park, and M. Naumov, "With Shared Microexponents, A Little Shifting Goes a Long Way," *ACM/IEEE 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
- [80] S. Sarangi and B. M. Baas, "DeepScaleTool: A Tool for the Accurate Estimation of Technology Scaling in the Deep-Submicron Era," *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2021.
- [81] M. Seo, X. T. Nguyen, S. J. Hwang, Y. Kwon, G. Kim, C. Y. Park, I.-H. Kim, J. Park, J. Kim, W. Shin, J.-H. Won, H. Choi, K. Kim, D. Kwon, C. Jeong, S. Lee, Y. Choi, W. Byun, S. Baek, H.-J. Lee, and J. Kim, "IANUS: Integrated Accelerator based on NPU-PIM Unified Memory System," *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.
- [82] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. J. Qiao, and P. Luo, "OmniQuant: Omnidirectionally Calibrated Quantization for Large Language Models," *International Conference on Learning Representations (ICLR)*, 2024.
- [83] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "RoFormer: Enhanced Transformer with Rotary Position Embedding," *Neurocomputing*, 2024.
- [84] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.
- [85] M. van Baalen, A. Kuzmin, S. S. Nair, Y. Ren, E. Mahurin, C. Patel, S. Subramanian, S. Lee, M. Nagel, J. B. Soriaga, and T. Blankevoort, "FP8 versus INT8 for efficient deep learning inference," *arXiv preprint arXiv:2303.17951*, 2023.
- [86] X. Wu, Z. Yao, and Y. He, "ZeroQuant-FP: A Leap Forward in LLMs Post-Training W4A8 Quantization Using Floating-Point Formats," *arXiv preprint arXiv:2307.09782*, 2023.
- [87] X. Wu, B. Zhou, Z. Gao, Y. Fu, Q. Zheng, Y. He, and H. Li, "KLLM: Fast LLM Inference with K-Means Quantization," *arXiv preprint arXiv:2507.23035*, 2025.
- [88] G. Xiao, J. Lin, M. Seznec, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," *International Conference on Machine Learning (ICML)*, 2023.
- [89] X. Xie, L. Wang, L. Xiao, M. Han, L. Liu, X. Xu, J. Wang, Z. Song, and X. Liao, "Amove: Accelerating LLMs through Mitigating Outliers and Salient Points via Fine-Grained Grouped Vectorized Data Type," *IEEE/ACM 58th International Symposium on Microarchitecture (MICRO)*, 2025.
- [90] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.
- [91] N. Yang, Z. Wang, Q. Sun, L. Lu, and F. Liu, "PISA: Efficient Precision-Slice Framework for LLMs with Adaptive Numerical Type," *ACM/IEEE Design Automation Conference (DAC)*, 2025.
- [92] Z. Yang, X. Jia, H. Li, and J. Yan, "LLM4Drive: A Survey of Large Language Models for Autonomous Driving," *arXiv preprint arXiv:2311.01043*, 2023.
- [93] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, "FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving," *Proceedings of Machine Learning and Systems (MLSys)*, 2025.
- [94] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, "Duplex: A Device for Large Language Models with Mixture of Experts, Grouped Query Attention, and Continuous Batch-ing," *IEEE/ACM 57th International Symposium on Microarchitecture (MICRO)*, 2024.
- [95] J. Zhang, J. Wei, P. Zhang, J. Zhu, and J. Chen, "SageAttention: Accurate 8-Bit Attention for Plug-and-play Inference Acceleration," *International Conference on Learning Representations (ICLR)*, 2025.
- [96] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit Quantization for Efficient and Accurate LLM Serving," in *Proceedings of Machine Learning and Systems (MLSys)*, 2024.
- [97] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.